

MATH471 — Introduction to Numerical Methods

Project

Ailin & Manuel — GC (Fall 2025)

Goals of the project

- Perform research beyond what is taught
- Select content and clearly organize ideas
- Orally present results in an appealing way
- Be critical on your results

1 Instructions

The goal of this project is to explore topics related to neural networks and see how they relate to the course content. In particular as neural networks can be used to approximate functions and the course introduces various strategies to achieve the same goal, we want to compare those approaches and better understand the contexts in which each is most suitable.

1.1 Format and due

Each of you needs to be in one and only one 3-member team for the project. The project will be graded according to the following three aspects:

1. General quality of the presentation:

- The presentation remains within the imparted time;
- The slide template is effectively used;
- The presenters do not read from the slides and focus on the audience;

2. Clarity of the explanations:

- The goals are clearly stated and reached;
- Results are properly highlighted and explained;
- The connections with the course content are comprehensively presented;
- The presentation does not feature any useless content, i.e. no slides are skipped during the presentation;

3. Quality of the application part:

- The comparisons between methods remain fair, e.g. put the same amount of efforts in the implementations and run them on the same hardware;
- Simple statistics about the running time of the methods are presented;
- Experimental results are discussed and conclusions are consistent with the results;

4. Questions and answers:

- Questions are answered in a clear way;
- All team members engage in the “discussion”;

Each of those four aspects have an equal weight such that their average leads to the project grade. All members are expected to work on equality, present slides, and answer questions. By default, all team members will receive the same project mark, however any non-participating student will receive deductions.

Presentations will be held on week 12. A team needs only to attend its own presentation. Slides as well as any source code must be submitted before the presentation time.

1.2 General expectations

The following section presents relevant topics to get your own research started on how to solve the problem stated in section 3. They are by no means the most rigorous proof, but it could provide you with the directions necessary to get started. We expect you to read **academic articles** as well as **textbooks**. We expect your presentation to effectively summarize the important aspects of your research on the problem within **15 minutes**; it will be followed by a **3-5 minutes** question and answer period. You **are not expected** you to present all aspects of what you have done during the research process. However, all group members should know very well what has been done so that anyone can answer if we choose to ask a question on a different aspect of your solution. If you select the "**Application**" part in tasks, we expect to see your code running properly; you will demonstrate it on your machine, making sure to focus on both accuracy and efficiency. Do not forget to **include your code in your submission**. As a rough guideline on the workload, we expect at least 2 hours of work from each student per week over three weeks.

2 Background on neural networks

Initially introduced in 1944 by McCulloch and Pitts, Neural Networks (NN) have now become a core component of Artificial Intelligence (AI) systems, meaning they are used in a wide variety of fields spanning much beyond AI and computer science. In this project our goal is twofold: understand the mathematics behind NN and compare their performance to that of more traditional numerical methods.

2.1 A brief introduction to neural networks

Neural Networks, as the name suggested, use multiple neurons to simulate the human brain. With the performance in proportion with the scale of the networks, they could be very powerful but may consume huge computational power. They were discarded in the 20th century, criticized for their bad performance compared with other statistical model such as support vector machine but regained the attention recently as the rapid development of GPU industry allows for large neural network model.

2.1.1 Mathematical definition

Formally a neural network is described as a tuple of four elements $(C, T, S(0), \Phi)$, where

- $C = (C_W, C_B, C_A)$ represents the constraints. $C_W \in \mathbb{R}$ stands for the weight, $C_B \in \mathbb{R}$ for the bias, and $C_A \in \mathbb{R}$ for the activity or neuron.
- $T = (F, I)$ is the topology of the space, which consists of the framework F , and the interconnection structure I .
- $S(0) = (S_W(0), S_B(0), S_A(0))$ is the initial state of the weight, bias, and activity.
- Φ is the transition function to proceed from one state to another.

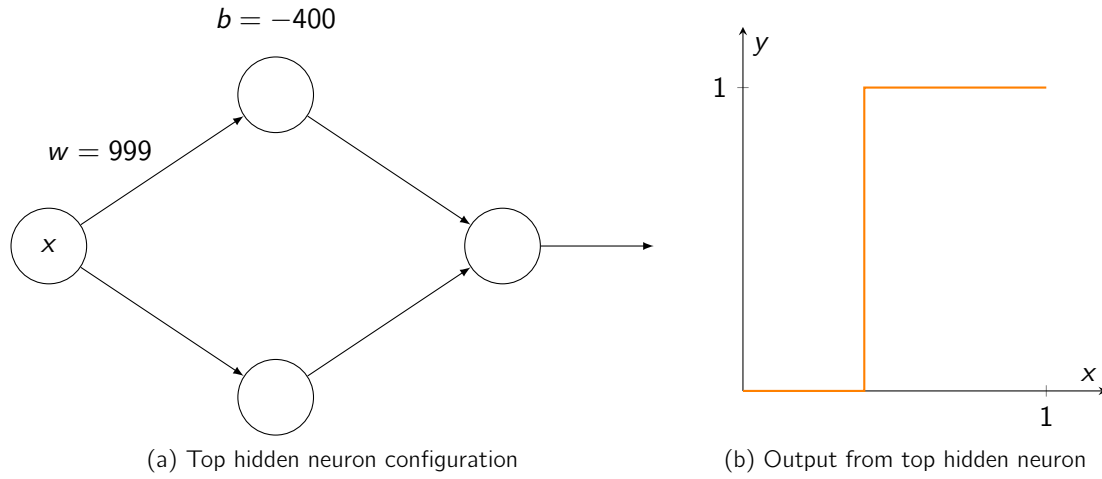


Figure 1: Neural network and its response

In this project, we restrict our attention to the scope of Multilayer perceptron (MLP), i.e. when the NN is in the form

$$f^{(n)} = \sigma_n(W_n f^{(n-1)} + B_n),$$

where σ_n is the activation function. In simple terms, it can be viewed as a non-linear function, W_n is a weight matrix, and B_n the bias vector. For a given multiple dimensional vector x , we first apply the linear transformation on it as $Wx + B$. Then we apply an activation function to it. We repeat the process for each layer and finally get $f^{(n)}$.

A key aspect of NN is the usage of a non-linear function. We illustrate it with a simple example where we use a *sigmoid* activation function in a two-layer NN,

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

The diagram of this simple two-layer neural network can be seen in Figure. 1. If we first consider the upper part, and set $w = 999$ and $b = -400$, we get

$$\sigma(x) = \frac{1}{1 + e^{-(999x-400)}},$$

which looks like a step function that turns at point $x_0 = -\frac{b}{w}$ (Figure 2). Keeping the weight as a large number τ , and use $s = -\frac{b}{w}$ to denote the neuron, we add the lower part and plug in the parameters $s_1 = 0.4, s_2 = 0.6, w_1 = 0.8, w_2 = -0.8$, to obtain a function $f_1 = \frac{w_1}{1 + e^{-\tau(x-s_1)}} + \frac{w_2}{1 + e^{-\tau(x-s_2)}}$, which looks like a bump function. The final output of this two-layer neuron is $\sigma(f_1)$.

As we can use the sigmoid function with a linear neuron to get a bump function, and any function can be approximated using a linear combination of bump functions, we have the feeling that neural network could potentially approximate any continuous functions. But what about other activation functions? For example, ReLU function

$$\sigma(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Remark. Can you think about some parameters to construct step function using ReLU function?

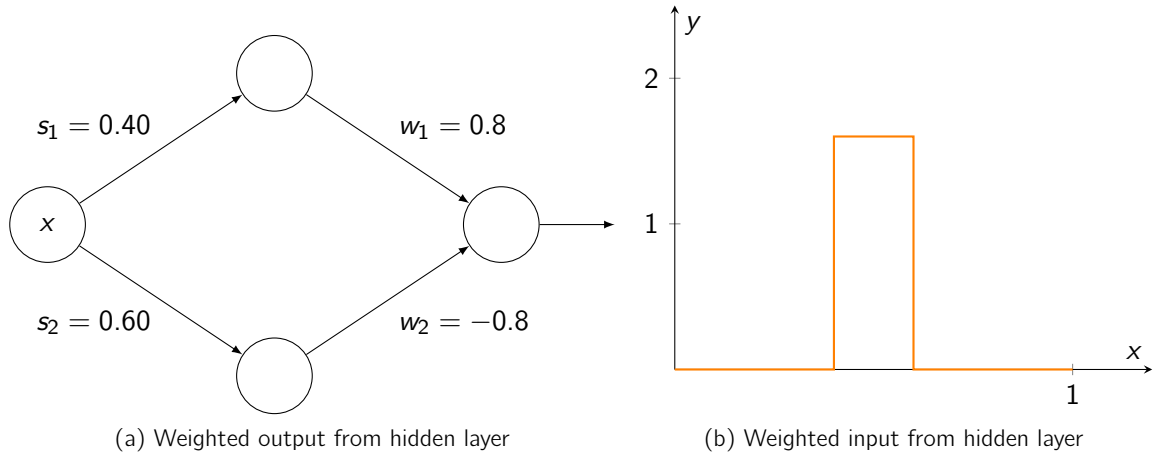


Figure 2: Use neural network to simulate a step function

2.1.2 Training of neural network

As we now understand the intuition behind why NN can be used to approximate functions, now we investigate the training process, namely, how to derive this approximation series. We can express the error, called *loss* in the context of neural networks, as

$$L_{\theta} = ||f^{(n)} - Y||$$

where Y is the “ground truth”. You can view it as the evaluation of the function we want to approximate; θ is the current parameters of the neural network. Our initial question is therefore to minimize this loss function with respect to the parameters (i.e. W_n, B_n) of the neural network. If the topological space is good enough, i.e. convex, we can safely apply the gradient descent algorithm. We know that L decreases fastest in the gradient direction so that we can modify the parameters by

$$W_n^{(k)} = W_n^{(k-1)} - \gamma \nabla L_{W_n^{(k-1)}} \text{ and } B_n^{(k)} = B_n^{(k-1)} - \gamma \nabla L_{B_n^{(k-1)}}.$$

The γ here is often taken as a constant bounding the step size at each iteration. It is called *learning rate* in machine learning. For $W_{n-1}^{(k)}$ and $B_{n-1}^{(k)}$, we proceed using the chain rule. Still, here we assume the convexity, which is rarely the case, especially in higher dimension. Therefore, in practice one has to be very careful when dealing with complicated neural networks as they may not converge as expected. A better understanding of the underlying mathematics can be helpful to avoid this type of issue.

The final strategy for updating the neural network could be summarized as follows.

1. Forward computation: calculate the loss using the current parameters of the neural network
2. Backward propagation: calculate the gradient of loss with respect to each parameter
3. Update: modify each parameter with certain learning rate. Return to step 1 and iterate.

Remark. As we want to use chain rule to calculate the gradient, what condition should the activation function satisfy?

2.1.3 Application of neural networks

In recent years, NN have become more and more powerful in various fields, including

- Computer Vision: Convolutional Neural Network, Vision transformer;
- Natural Language Processing: ChatGPT, ChatGLM;
- Decision-Making: AlphaGo, AlphaZero;
- Time series: Recurrent Neural Network, LSTM;
- Multimodal: CLIP, Dall-E;

These models exhibit different structures to deal with different type of input and output data, but the inner core always remains the same.

2.2 The mathematics of neural networks

Similarly to how Numerical Analysis finds its roots in Stone-Weierstrass theorem and more generally in topology, NN heavily rely on topology in order to approximate any real valued continuous function. This result is often referred to as *Universal Approximation Theorem*.

2.2.1 Digression on general topology

In chapter 2 we briefly went through key concepts and results from topology in the context of metric spaces, hence introducing all results needed at a later stage of the course. We emphasized the importance of continuous maps which play a similar role as isomorphisms in algebra and of the choice of opens which can lead to various topologies, some being “better” than others. This point is especially critical in the more general case where we do not have access to a norm or distance.

When X is a normed vector space, $(X, \|\cdot\|_X)$ is automatically endowed with a topology called *strong topology* where the opens are defined as in the course, i.e. the open balls $B(x, r) = \{y \in X : \|y - x\| < r\}$. The major benefit is that in such a space a sequence $(x_n)_{n \in \mathbb{N}}$ converges in norm to x , if and only if $\|x - x_n\|$ tends to 0 as n tends to infinity.

While strong topology is “convenient” as it allows for uniform convergence when considering the supremum norm, it fails to capture other types of convergence such as point-wise convergence, convergence in measure, smooth convergence, or convergence on compact subsets. Interestingly the phrasing “strong topology” is opposed to “weak topology” which renders the feeling of “how many opens and compact does a topology have”. In fact, we are often most interested in the “nice” properties exhibited by compacts and the weak topology will provide “fewer opens”, hence making compacts “more common”. This is of utmost importance for instance in the case of deriving numerical methods to solve partial differential equations where only weaker topologies can be employed to leverage compact related results. The goal is then to either work on these weaker topologies or derive initial results on them and then “upgrade” to stronger modes of convergence.

More specifically, the concept of weak topology on a vector space X is related to the notion of dual space, i.e. the set of the linear forms on X . The basic idea is to “limit” the number of opens by including as few as possible while keeping the evaluation map $x \mapsto f(x)$ continuous for all continuous $f \in X^*$. Therefore, dual spaces play a central role in algebraic topology, i.e. the study of topological vector spaces.

2.2.2 The universal approximation theorem

In slightly more technical terms, the universal theorem states that any continuous function f on a compact can be uniformly approximated by a feedforward neural network having only a single hidden layer. While we are not providing a complete formal proof of this result we explain the mainlines of how to think about it.

We denote by I_n the set $[0, 1]^n$ and by $\mathcal{F}(I_n, \mathbb{R})$, the vector space of the functions from I_n to $\mathbb{R}$. We immediately notice the following two properties: by Heine-Borel ?? I_n is compact and as for (+) the set of the continuous functions $\mathcal{C}(I_n)$ is closed in $\mathcal{F}(I_n, \mathbb{R})$ and contains 0, $\mathcal{C}(I_n)$ is a vector space. Endowed with the supremum norm, $\mathcal{C}(I_n)$ becomes a topological vector space for which we can define $\mathcal{C}^*(I_n)$ the dual of $\mathcal{C}(I_n)$, composed of the linear functions defined on $\mathcal{C}(I_n)$.

Interestingly we now meet all the requirements to apply *Riesz Representation Theorem* which states as follows.

Theorem 1 (Riesz Representation). *Any bounded linear functional φ on a compact U can be represented by a unique regular signed Borel measure μ such that*

$$\varphi(f) = \int_U f d\mu.$$

In simple terms it means that any linear functional can be “replaced” by an integral with respect to a well “behaving” measure. This can be seen from the definition of a *regular measure*, i.e. a measure verifying

- $\mu(K) < \infty$ for any compact K ;
- $\mu(E) = \sup\{\mu(K) : K \subseteq E, K \text{ compact}\};$
- $\mu(E) = \inf\{\mu(U) : E \subseteq U, U \text{ open}\};$

Intuitively those three properties convey the feeling of “regularity”, as

- the measure of a compact is finite;
- the measure of E is given by
 - the smallest measure of opens containing E ;
 - the largest measure of compacts contained in E ;

Further thinking in terms of measure μ we introduce the notion of *discriminatory* function.

Definition 2. *A continuous function σ on I_n is said to be discriminatory, if $\mu = 0$ is the only signed regular Borel measure such that for $w, x \in \mathbb{R}^n$ and $b \in \mathbb{R}$,*

$$\int_{I_n} \sigma(w^\top x + b) d\mu = 0.$$

As σ takes an affine transformation as input, we could easily imagine such function as “losing important information”. For instance σ could map the affine space $w^\top x + b$ to a set with null μ measure. A discriminatory function σ would never do that.

We now consider a finite linear combination of discriminatory function, i.e. we construct the function

$$G(x) = \sum_{i=0}^d \alpha_i \sigma(w_i^\top x + b_i). \quad (1)$$

At this stage we “hope” that linear combinations of discriminatory functions defined by (1) can approximate

any continuous function on I_n or stated otherwise, that the set S of the linear combinations G is dense in $C(I_n)$. Therefore, calling S the set of all linear combinations G , we want to show that the closure $\overline{S}$ of S is exactly $C(I_n)$. Reasoning by contradiction we assume that this is not the case, i.e. $\overline{S} = R \subsetneq C(I_n)$ and consider the following key result.

Theorem 3 (Hahn-Banach). *Let W be a subspace of a normed vector space V and $x_0 \in V$. Then x_0 is in the closure $\overline{W}$ of W , if and only if there is no bounded $\varphi \in V^*$ which is 0 everywhere on W while $\varphi(x_0) \neq 0$.*

Hence, we can find a non-zero $\varphi \in C(I_n)^*$ such that $\varphi(S) = \varphi(R) = 0$. Then by Riesz Representation (theorem 1), we can express φ as

$$\varphi(f) = \int_{I_n} f \, d\mu,$$

where μ is a signed regular Borel measure. In particular in our case where using a discriminatory function σ we obtain for all w and b

$$\int_{I_n} \sigma(w^\top x + b) \, d\mu = 0.$$

This directly yield a contradiction as μ is not supposed to be null. As a result $R = C(I_n)$ and the set S of the linear combinations G is dense in $C(I_n)$.

Looping back to neural networks, we understand that the discriminatory function σ corresponds to the sigmoid function which can be “weighed” with w and shifted with b in order to “resemble” a step function (section 2.1).

3 Tasks

In this project we want to allow you to choose what to explore! Here is a non-exhaustive list of topics that you might want to investigate.¹ You must choose one topic from the list below. You need to first complete the “Theory” part of a topic before moving on the “Application” part.

1. Pure mathematics.

Theory. Dig deeper into the mysteries of topology, measure theory, and linear algebra with the goal further understanding the key results discussed in section 2.2. This could for instance include notions on Hausdorff space, algebraic and topological dual spaces, as well as Radon, Haar, Lebesgue, and Borel measures.

Application. A very recent result² shows that the Universal Approximation Theorem can be generalized to uniform convergence on the whole $\mathbb{R}^n$ which is not compact. How is this working, what are the technical challenges faced to reach this result and how useful is it in practice?

2. Activation functions.

Theory. Why is the sigmoid function discriminatory? Is the ReLU function mentioned in 2.1 discriminatory? What other discriminatory functions could be used? Are some functions better than others and why?

Application. Please follow the tutorial script to implement the forward computation and back propagation function.³ Use the dataset (`p1-set1.txt`) to compare regular approximation

¹If you have another topic idea, please contact us, and we can discuss whether it is possible to include it.

²*Noncompact uniform universal approximation*, Teun and van Nuland.

³The tutorial is modified from Stanford CS231n course assignment

methods with the NN approach. In particular, study the impact of the activation function on the NN performance.

3. Multivariate approximation.

Theory. Extend the neural network as well as the interpolation method to higher dimensions. Are there any extra conditions that need to satisfy?

Application. Approximate the given multiple dimension dataset with neural network and interpolation method (`p1-set2.txt`), compare their performance. `p1-set2.txt` is a 2-dimensional approximation of the loss-landscape topology for a fully connected neural network trained on MNIST. This approximation is accomplished by sampling two random direction vectors in parameter space, and computing the normalized loss at a number of points on the plane defined by the two random vectors.

You might find the following script useful for data exploration.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 STEPS=100
4 lf = np.loadtxt('p1-set2.txt')
5 fig = plt.figure()
6 ax = plt.axes(projection='3d')
7 X = np.array([[j for j in range(STEPS)] for i in range(STEPS)])
8 Y = np.array([[i for _ in range(STEPS)] for i in range(STEPS)])
9 ax.plot_surface(X, Y, lf, rstride=1, cstride=1,
10               cmap='viridis', edgecolor='none')
11 fig.show()
```

4. Integration.

Theory. As proven in 2.2, neural network could approximate the function “in shape”, so we could also do integration using neural network. Research more on it and discuss what could be the advantages and disadvantages of this strategy?

Application. Do a simple integration task on the provided dataset and test your result. Compare the performance with the method we introduced in class.

5. Differential equations.

Theory. We have learned in MATH286 or MATH256 that we can approximate the solution of ODE using power series. In this course, we learn some other numerical methods such as Runge-Kutta method. Having similar property, could neural networks be used to solve ODE in numerical ways? Research more and give the intuitive idea why we can do it. Also try to give the loss function in this case.

Application. Compare various strategies to solve the ODE

$$x^2 \frac{d^2 y}{dx^2} + x \frac{dy}{dx} + x^2 y = 0,$$

with an initial value of your choice. For instance, on top of Runge-kutta also investigate Adams-Bashford and Adams-Moulton methods. Compare their performance and accuracy to the ones of neural networks with different activation functions.